Homework 8

Part 1: Backend Service (5 points)

Implement a Kafka-based user management service that handles the following operations:1. CREATE_USER – Create a new user

- **Input:** `{ "operation": "CREATE_USER", "name": string, "email": string, "age": number }`
- **Output:** `{ "success": true, "userId": string, "message": "User created" }`

2. GET_USER – Retrieve user by ID

- **Input:** `{ "operation": "GET_USER", "userId": string }`
- **Output:** `{ "success": true, "user": { userId, name, email, age } }`

Kafka Requirements

- **Use two topics:**
  - `user_requests`
  - `user_responses`
- **Each message must include:**

```
None
{
    "correlation_id": "unique-id",
    "reply_to": "user_responses",
    "data": { ... }
}
```

- **The worker must:**
  - Consume from `user_requests`
  - Send response to `reply_to`
  - Include same `correlation_id` in response

General Requirements

- Store users **in-memory** (Python dictionary)
- Generate unique user IDs (**UUID**)

- Validate input (email format, age > 0)
- Return proper error messages for invalid operations

Demonstration

Create a file `homework_demo.py` that demonstrates operations in sequence:

- CREATE
- GET_USER

Screenshots Requirements:

1. **Docker Containers Running** - Command: `docker ps`
    - Worker (consumer) service started (shows: "Consumer is ready and listening…")
    - FastAPI service running (shows server started on `[http://127.0.0.1:8000](http://127.0.0.1:8000)`)
2. **Execute CREATE_USER and GET_USER operations** (from `homework_demo.py` or API call)
3. **Show logs of:**
    - message sent to `user_requests` topic
    - message received and processed by worker
    - response sent to `user_responses` topic
    - response received by FastAPI and returned to client

Part 2: 3-Agent Mini-System with Kafka + LangChain

Build a tiny system with 3 agents that communicate through Kafka:

1. **Planner**: Makes a plan for the question.
2. **Writer**: Writes a short answer using LangChain.
3. **Reviewer**: Checks the answer and approves it.

All messages must go through Kafka topics. No agent should call another agent directly. You may reuse agents from your previous work.System Workflow

Sender	Reads Topic	Sends Message	To Topic
You	(N/A)	A question	`inbox`

Planner	`inbox`	A TODO plan	`tasks`
Writer	`tasks`	A draft answer	`drafts`
Reviewer	`drafts`	An approval	`final`

You will run each agent in its own terminal.Kafka Topic Creation

Create four topics: `inbox`, `tasks`, `drafts`, and `final`.

(Use your Kafka CLI or UI; example CLI commands:)

```
None
   kafka-topics --create --topic inbox --partitions 1
--replication-factor 1 --bootstrap-server localhost:9092
   kafka-topics --create --topic tasks --partitions 1
--replication-factor 1 --bootstrap-server localhost:9092
  kafka-topics --create --topic drafts --partitions 1
--replication-factor 1 --bootstrap-server localhost:9092
   kafka-topics --create --topic final --partitions 1
--replication-factor 1 --bootstrap-server localhost:9092
```

Running the Agents

Run each agent in a separate terminal:

Terminal	Command
Terminal 1	`python planner.py`
Terminal 2	`python writer.py`
Terminal 3	`python reviewer.py`
Terminal 4	`python send_question.py` (Send one question)

Viewing the Result

Read the final result from the `final` topic.

You should see a JSON with `"status": "approved"` and an `"answer"`.What to Submit

- Screenshot or log lines showing each agent received and sent messages.
- Screenshot of the message in topic `final` with `"status": "approved"`.
- A short report explaining how Kafka helped your agents coordinate.